The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Student Name : ADITI RAJ

Registration Number : 24BAI10085

Slot : A24

Date of Submission : 31/03/2026

Part A — Origin and Philosophy

A1. The Origin Story: More Than Just a Scripting Language

- **The Frustration:** The "ABC" Gap In the late 1980s, Guido van Rossum was employed at the Centrum Wiskunde & Informatica (CWI), a computer institute in the Netherlands. During this period, software developers had a frustrating choice between two alternatives:
  Low-level languages (such as C): These are powerful and fast but require a tremendous amount of work to code. Simple tasks require writing hundreds of lines of code.
  High-level languages (such as ABC): Guido van Rossum had already worked on a language called ABC. ABC was a beautiful language that was easy to understand for people who are not programmers. However, it was a "closed" language that could not talk to the operating system or handle complex file systems.

The Christmas Project (1989) In the Christmas week of 1989, Guido van Rossum was stuck at home because his office was closed during the holiday. He wanted a "hobby" project to keep him occupied during the week. He decided to write an interpreter for a new scripting language that would be a descendant of ABC but would also appeal to Unix/C hackers.

- **Why Share it?** (The Open Source Choice)Why did Guido choose to share his creation rather than keep it proprietary? Guido had a very down-to-earth reason for choosing to make his creation available to the world. He wanted his creation to live on. By making it

available to the world in 1991, he enabled the world to improve it and extend it in ways he might not have been able to. A programming language is only as good as the community of people who support it. Guido realized that if he had not shared his creation, it would have died in the Netherlands.
The Philosophical IdentityPython is not like most programming languages in the world. Guido had a philosophy for the language, called "The Zen of Python." This philosophy is to make the language as readable as possible for the human eye, rather than the machine.

- **The Problem it Solved:** It solved the problem of programmer burnout.
The 'Monty Python' ConnectionTo keep things light and fun, Guido named his new language after the popular 1960s-1970s British TV show and film series 'Monty Python's Flying Circus.' Guido wanted to make the point that programming should not be a boring, corporate task. Programming should be fun.

A2. The License — What it Actually Says

- **The Four Freedoms and the PSF License**

The "Four Freedoms" established by the Free Software Foundation are the heart of the open source movement:

  - Freedom 0: The freedom to run the program as you wish, for any purpose.
  - Freedom 1: The freedom to study how the program works and change it.
  - Freedom 2: The freedom to redistribute copies so you can help others.
  - Freedom 3: The freedom to distribute copies of your modified versions to others.

    Are these freedoms satisfied by Python? Yes, they are. The PSF License is a "permissive" royalty-free license, and it specifically permits you to use, copy, modify, and distribute Python, either in its source or binary form.

- **Modification, Sale, and Sharing Changes**

Can a company modify Python, sell it, and not have to share their modifications with anyone else? Yes, it can.
The PSF License is "permissive" rather than "copyleft," just like the MIT license, and does not have a copyleft provision. This means a company like Google or Meta can modify the Python interpreter for their internal servers, make a lucrative product on top of it, and not be forced by law to release their source code modifications publicly. The only significant requirements are that they have to include the original copyright notice and license text from the Python Software Foundation within their documentation.

- **GPL vs. MIT Licensing (and Python's PSF)**
- **GPL (General Public License): It** is a "Copyleft" License. If you modify GPL software (for example, Linux Kernel) and distribute it, you must also share your modifications under the GPL License.

MIT/PSF License: It is a "Permissive" License. It allows maximum distribution with minimal restrictions.
My Choice: Since I am a student of AI/ML, my choice would be a permissive license like the PSF License or MIT License. It would encourage maximum distribution of my code by both scholars and business organizations, who might use it in their systems without any fear of "leakage" of their proprietary code.

- **'Free as in Beer' vs. 'Free as in Freedom'**

Free as in Free Beer (Gratis): It means "free" in the context of money. There is no need to spend a single Rupee to download and use Python.

Free as in Freedom (Libre): It means freedom from certain rules and regulations. It means that you are free to open the code of the Python program and see how the print() function works and even modify it if you do not like it.

Example from Python: Python is distributed for free (Free Beer). However, the freedom aspect of Free Software is demonstrated by projects like PyPy and MicroPython. Since the code was Free as in Freedom, it was possible for the developers to take the Python code and develop a completely new version of Python running on microchips or running faster than the original Python. It would not be possible with a proprietary software like MATLAB.

A3. The Ethics of Open Source

- **Should all software be open source ?**

The Argument "For": Software is a tool for modern living. The restriction of human knowledge by not making the 'blueprint' of the tool available is a violation of human knowledge. Were it not for the open-source nature of the Python language, the world might not have experienced the AI/ML revolution so quickly and might have been restricted to only those who could pay the high licensing fees.

The Argument "Against": While the world might be a better place if all software were open source, the reality is that such a requirement might stifle economic incentive. Companies need to have 'trade secrets' to justify the high R&D costs of specialized software.

- **Ethicality of Profitable Products on Free Labor**

The Dilemma: Google and Meta, the world's most valuable companies, utilize the Python language and the Linux kernel to operate their global infrastructure without paying a 'user fee' to the original creators.The Ethical Defense: This is ethical behavior because the companies give back to the community by paying the original creators to work for them or by contributing back to the community by open-sourcing their own tools.
The Critical View: Some call it "digital sharecropping," wherein a small number of multi-billion dollar corporations reap disproportionate rewards from the enthusiasm and hard work of thousands of volunteers who will never see a penny of the profit.

- **'Standing on the Shoulders of Giants' and Innovation**

Meaning: In the context of software development, this means that no programmer ever truly starts from scratch. If you write a Python script for artificial intelligence, you are literally "standing on the shoulders of giants" because of the decades of work done by the Python Foundation, the Linux Kernel team, and C compiler authors.
Enabling Innovation: Open source is a "force multiplier." Rather than "reinventing the wheel" (like a web server or a sorting algorithm), programmers can now focus on "inventing the car" (new forms of artificial intelligence applications).
Undermining Innovation? The counter-argument is that if a "good-enough" open-source solution exists for a given problem, then programmers will not innovate and invent a fundamentally "better" but different proprietary solution.

Part B: Linux Footprint

- ## Installation Method

  Apt Package Manager: Python is pre-installed on most Linux distributions such as Ubuntu or Debian. You can update or install it using the following command: sudo apt install python3.
  Source Compilation: At times, developers need Python for specific AI/ML purposes; they download the source code from python.org and use make to compile it for optimal performance on their hardware.

- ## Key Directories

  Binaries (/usr/bin/python3): This is the directory where the Python interpreter is stored in the form of an executable script.
  Standard Libraries (/usr/lib/python3.x): This directory contains the library of modules that makes Python work.
  Site-Packages (/usr/local/lib/python3.x/dist-packages): This directory contains the external libraries you install using pip for your AI/ML needs.

- ## User and Group

Execution Identity: When you execute a Python script, it will run with your username.

Security Significance: This is a critical security feature since your script will run as a standard user and not "root" or "admin" user. Thus, even if your Python script has a bug that causes it to execute a function or method it was not supposed to execute, it will not have the permissions to change critical settings or delete critical files.

Service Management

Interpreter Nature: Unlike a web server such as Apache or a database such as MySQL, Python does not need to be running in the background since it is an interpreter.

Custom Services: In case you need a background script for your AI/ML needs, you can use systemd to manage it.
Some of the commands you could use include:
sudo systemctl start my_python_script.service
sudo systemctl stop my_python_script.service

- ### Update Model

  Community Patches: When a security vulnerability is discovered in Python by the Python Software Foundation (PSF), a patch is released.
  Distribution Pipeline: These patches are then picked up by your Linux distribution maintainers and pushed to your computer via the repository. You receive these patches when you use the following commands: sudo apt update && sudo apt upgrade.

Part C — The FOSS Ecosystem

Dependencies (What Python needs to run)

No software is an island. In order for Python to run on a Linux platform, it needs a few more **FOSS tools to be installed:**
The C Compiler: The standard Python interpreter is written in C and needs the GNU Compiler Collection to be compiled and optimized for different hardware.
GNU C Library: Python needs this library to perform basic operations such as file operations, memory management, and handling the system clock.
OpenSSL: When it comes to secure network operations such as pip install or downloading a package over a secure connection or HTTPS, Python needs OpenSSL.
SQLite: Python comes bundled with SQLite, a popular open-source database library, which makes it a breeze for Python to work with databases.

Inspiration and Spin-offs (What Python enabled)

Python's syntax has inspired many more projects and "spin-offs":
MicroPython/CircuitPython: These are "spin-offs" of Python that are optimized for running on microcontrollers such as Arduino or ESP32.
PyPy: This is a "spin-off" of Python that has implemented a "Just-In-Time" compiler to make Python programs run much faster than the standard interpreter.
AI/ML Frameworks: Python has enabled the development of open-source libraries such as TensorFlow and PyTorch for Artificial Intelligence by Google and Meta Platforms, respectively, which have revolutionized the field of Artificial Intelligence.

Role in the Web and LAMP Stack

Python plays an important role in the web development arena:
The "P" in the LAMP Stack:
Although the "P" in the stack originally referred to the programming language Perl or PHP, Python has become the de facto standard for this position in the stack.
Frameworks:

Python offers several frameworks, such as Django and Flask, which allow the creation of complex, secure, and open-source web applications.

Automation:

In the web arena, Python acts as the glue that binds together different services, databases, and applications.

Community and Governance

Python follows a community-based approach, not a corporate one:

Python Software Foundation (PSF):

This is a non-profit organization that manages the intellectual property rights for Python.

PEPs (Python Enhancement Proposals):

This is the primary mechanism for proposing enhancements. Anyone in the community can write a proposal, which is then discussed in the community.

Communication Channels:

The Python community interacts through mailing lists like python-dev, the global PyCon conferences, and online communities like the Reddit community r/Python or Discord.

Historical Governance:

Python was originally managed by Guido van Rossum, the Benevolent Dictator for Life (BDFL), but now it is managed by a Steering Council of five elected members.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparison Table: Python vs. MATLAB :

Dimension	Python (Open Source)	MATLAB (Proprietary Alternative)
Cost	**Free ($0)**. Available to everyone, from students to large corporations, without licensing fees.	**High Cost**. Requires expensive annual licenses, with additional costs for specific "toolboxes."
Security (Auditability)	**High**. Anyone can audit the source code. Vulnerabilities are often found and patched quickly by the global community.	**Low/Closed**. Only MathWorks (the company) can audit the code. Users must trust the company's internal security team.

Support and Reliability	**Community-Driven**. Relies on massive forums like StackOverflow and GitHub. Very reliable due to widespread use in production.	**Corporate Support**. Offers dedicated technical support lines and guaranteed service-level agreements (SLAs).
Freedom to Modify	**Full Freedom**. Users can modify the interpreter itself (e.g., creating PyPy) or fix bugs locally.	**No Freedom**. Users are legally and technically barred from modifying the underlying source code.
Control	**Community Control**. Governed by the PSF; features are decided through public PEPs (Python Enhancement Proposals).	**Corporate Control**. The roadmap and feature sets are decided strictly by MathWorks based on profit and market goals.

Overall Verdict

Considering the depth of this audit, I would highly recommend Python for use in a real-world scenario, particularly for AI and Machine Learning domains. Although MATLAB provides top-notch support for control systems, Python's freedom from licensing costs makes it a better option for a production environment running on the cloud. Its massive collection of libraries like TensorFlow and Scikit-learn ensures that it is at the forefront of innovation, well ahead of the pace at which a single corporation can develop.

I would absolutely contribute to the Python ecosystem. As a B.Tech student, contributing to open-source libraries is not just about coding; it is about being part of a culture of knowledge sharing. Whether it is about documenting a library that might be of interest to a handful of people or fixing a bug in an AI library, contributing to the Python ecosystem lets me "stand on the shoulders of giants" and contribute to building the foundation for a new generation of programmers.

Shell Script Tasks

Script 1 — System Identity Report :

Code :

```
aditi@Aditi: ~
  GNU nano 7.2                                    system_report.sh *
#!/bin/bash
# Script 1: System Identity Report
# Author: aditi | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="aditi"
SOFTWARE_CHOICE="Python"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)
DISTRO=$(lsb_release -ds 2>/dev/null || cat /etc/*release 2>/dev/null | head -n1 || uname -s)
PYTHON_VERSION=$(python3 --version 2>/dev/null)

# --- Display ---
echo "========================================"
echo "Open Source Audit - $STUDENT_NAME"
echo "Software Choice: $SOFTWARE_CHOICE"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Directory: $HOME_DIR"
```

```
aditi@Aditi: ~
  GNU nano 7.2                                    system_report.sh *
# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)
DISTRO=$(lsb_release -ds 2>/dev/null || cat /etc/*release 2>/dev/null | head -n1 || uname -s)
PYTHON_VERSION=$(python3 --version 2>/dev/null)

# --- Display ---
echo "========================================"
echo "Open Source Audit - $STUDENT_NAME"
echo "Software Choice: $SOFTWARE_CHOICE"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "User         : $USER_NAME"
echo "Home Directory: $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Current Date : $CURRENT_DATE"
echo "Python Version: $PYTHON_VERSION"
echo "----------------------------------------"
echo "This operating system is covered by the"
echo "GNU General Public License (GPL)"
echo "========================================"
```

Output:

```
PS C:\WINDOWS\System32> wsl
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

aditi@Aditi:/mnt/c/WINDOWS/System32$ aditi@Aditi:~$
aditi@Aditi:~$: command not found
aditi@Aditi:/mnt/c/WINDOWS/System32$ nano system_report.sh
aditi@Aditi:/mnt/c/WINDOWS/System32$ cd ~
aditi@Aditi:~$ pwd
/home/aditi
aditi@Aditi:~$ nano system_report.sh
aditi@Aditi:~$ chmod +x system_report.sh
./system_report.sh
==========================================
Open Source Audit - aditi
Software Choice: Python
==========================================
Distribution : Ubuntu 24.04.4 LTS
Kernel       : 6.6.87.2-microsoft-standard-WSL2
User         : aditi
Home Directory: /home/aditi
Uptime       : up 1 hour, 34 minutes
Current Date : Mon Mar 30 18:28:56 UTC 2026
Python Version: Python 3.12.3
------------------------------------------
This operating system is covered by the
GNU General Public License (GPL)
==========================================
aditi@Aditi:~$ |
```

Explanation :This script will display a welcoming screen with system information. The variables and command substitution (`uname -r`, `whoami`, `$HOME`, `uptime -p`, `date`) are used. The distribution name is also fetched from `lsb_release` or `/etc/os-release`. This script is an example of basic concepts in shell scripting. Variables, command substitution with `$()`, and formatting with `echo` are used. The license is hardcoded as it is open-source and Ubuntu is under GPL.

Script 2 — FOSS Package Inspector

Code:

```
aditi@Aditi: ~
  GNU nano 7.2                                package_inspector.sh
chmod +x package_inspector.sh
./package_inspector.sh
#!/bin/bash
# Script 2: FOSS Package Inspector
# Description: Checks if Python is installed, displays version info, and shows package description

# Define the package to check
PACKAGE="python"

echo "========================================"
echo "FOSS Package Inspector"
echo "Checking package: $PACKAGE"
echo "========================================"
echo

# Check if package is installed using rpm
if rpm -q "$PACKAGE" &>/dev/null 2>&1; then
    echo "✓ $PACKAGE is INSTALLED on this system."
    echo
    echo "--- Package Details ---"
    # Get detailed package information
    rpm -qi "$PACKAGE" 2>/dev/null | grep -E 'Name|Version|Release|Architecture|License|Summary|Description' | head -10
    echo "-----------------------"
    echo
```

```
aditi@Aditi: ~
GNU nano 7.2                                    package_inspector.sh *
    echo
    echo "--- Package Details ---"
    dpkg -l "$PACKAGE" 2>/dev/null | grep "^ii"
    echo
    echo "--- Python Version Details ---"
    python3 --version
    echo
else
    echo "X $PACKAGE is NOT INSTALLED on this system."
    echo "To install: sudo apt install python3"
    echo
fi

echo "--- About Python ---"
case $PACKAGE in
    python3)
        echo "Python: A high-level, interpreted programming language known for its simplicity."
        echo "Used for web development, data science, automation, and more."
        ;;
esac

echo
echo "========================================"
echo "Inspection complete."
echo "========================================"
```

Output:

```
aditi@Aditi:~$ pwd
/home/aditi
aditi@Aditi:~$ nano package_inspector.sh
aditi@Aditi:~$ chmod +x package_inspector.sh
aditi@Aditi:~$ ./package_inspector.sh
========================================
FOSS Package Inspector
Checking package: python3
========================================

✓ python3 is INSTALLED on this system.

--- Package Details ---
ii  python3        3.12.3-0ubuntu2.1 amd64        interactive high-level object-oriented language (default python3 versi
on)

--- Python Version Details ---
Python 3.12.3

--- About Python ---
Python: A high-level, interpreted programming language known for its simplicity.
Used for web development, data science, automation, and more.

========================================
Inspection complete.
========================================
aditi@Aditi:~$
```

AI-generated, for reference only

Explanation :This script checks if a software package is installed by checking if it is an RPM-based system or a DPKG-based system with an `if` statement. The script then uses a `case` statement to print out a description of the package's purpose. The script covers basic concepts like `if-then-else`, `case`, and pipes with `grep`.

Script 3 — Disk and Permission Auditor

Code:

```
aditi@Aditi: ~
  GNU nano 7.2                                    disk_auditor.sh *
#!/bin/bash
# Script 3: Disk and Permission Auditor

# Define array of directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "Directory Audit Report"
echo "Date: $(date)"
echo "========================================"
echo

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Get permissions, owner, and group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Get size in human readable format
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR => Directory does NOT exist"
    fi
done

echo
echo "========================================"
```

```
aditi@Aditi: ~
GNU nano 7.2                                    disk_auditor.sh *
 Define array of directories
IRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

cho "========================================"
cho "Directory Audit Report"
cho "Date: $(date)"
cho "========================================"
cho

or DIR in "${DIRS[@]}"; do
   if [ -d "$DIR" ]; then
       # Get permissions, owner, and group
       PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
       # Get size in human readable format
       SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
       echo "$DIR => Permissions: $PERMS | Size: $SIZE"
   else
       echo "$DIR => Directory does NOT exist"
   fi
one

cho
cho "========================================"
cho "Audit Complete"
cho "========================================"
```

Output:

```
Inspection complete.
========================================
aditi@Aditi:~$
aditi@Aditi:~$ nano disk_auditor.sh
aditi@Aditi:~$ chmod +x disk_auditor.sh
aditi@Aditi:~$ ./disk_auditor.sh
========================================
Directory Audit Report
Date: Mon Mar 30 19:18:12 UTC 2026
========================================

/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 82M
/home => Permissions: drwxr-xr-x root root | Size: 52K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 20K

========================================
Audit Complete
========================================
aditi@Aditi:~$
```

Explanation :This script utilizes a `for` loop to traverse through various critical directories such as `/etc`, `/var`, `/home`, and `/usr`, displaying disk space with `du -sh`, as well as permissions/owner with `ls -ld`. It covers various programming concepts such as `for` loops, executing commands, and using `awk` or `cut`.

Script 4 — Log File Analyzer

Code:

aditi@Aditi: ~

GNU nano 7.2 log_analyzer.sh *

```bash
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/syslog [keyword]
# Description: Analyzes log files for specific keywords

# Get command line arguments
LOGFILE=$1
KEYWORD=${2:-"error"}    # Default keyword is 'error' if not provided
COUNT=0
MATCHING_LINES=()

# Check if log file is provided
if [ -z "$LOGFILE" ]; then
    echo "Error: No log file specified."
    echo "Usage: $0 <logfile> [keyword]"
    echo "Example: $0 /var/log/syslog error"
    exit 1
fi

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

echo "========================================="
```

aditi@Aditi: ~

GNU nano 7.2 log_analyzer.sh *

```bash
fi

echo "========================================="
echo "LOG FILE ANALYZER"
echo "========================================="
echo "File: $LOGFILE"
echo "Keyword: $KEYWORD"
echo "========================================="
echo

# Check if file is empty (do-while style retry)
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: $LOGFILE is empty."

    # Retry logic - wait for file to have content
    RETRY_COUNT=0
    MAX_RETRIES=3

    while [ ! -s "$LOGFILE" ] && [ $RETRY_COUNT -lt $MAX_RETRIES ]; do
        RETRY_COUNT=$((RETRY_COUNT + 1))
        echo "Attempt $RETRY_COUNT: File is empty. Waiting 2 seconds..."
        sleep 2

        # Check again if file has content
        if [ -s "$LOGFILE" ]; then
            echo "✓ File now has content. Continuing..."
```

```
aditi@Aditi: ~
GNU nano 7.2                                    log_analyzer.sh *
echo

# Check if file is empty (do-while style retry)
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: $LOGFILE is empty."

    # Retry logic - wait for file to have content
    RETRY_COUNT=0
    MAX_RETRIES=3

    while [ ! -s "$LOGFILE" ] && [ $RETRY_COUNT -lt $MAX_RETRIES ]; do
        RETRY_COUNT=$((RETRY_COUNT + 1))
        echo "Attempt $RETRY_COUNT: File is empty. Waiting 2 seconds..."
        sleep 2

        # Check again if file has content
        if [ -s "$LOGFILE" ]; then
            echo "✓ File now has content. Continuing..."
            break
        fi
    done

    # If still empty after retries
    if [ ! -s "$LOGFILE" ]; then
        echo "Error: File remains empty after $MAX_RETRIES attempts. Exiting."
        exit 1
```

Output:

```
aditi@Aditi: ~
==========================================
aditi@Aditi:~$ ./log_analyzer.sh /var/log/syslog failed
==========================================
LOG FILE ANALYZER
==========================================
File: /var/log/syslog
Keyword: failed
==========================================

Analyzing log file...

==========================================
ANALYSIS RESULTS
==========================================
Keyword 'failed' found 78 times in /var/log/syslog

Total lines in file: 3565
Percentage of lines containing 'failed': 2.18%

==========================================
LAST 5 MATCHING LINES
==========================================
The last 5 lines containing 'failed':

  • 2026-03-30T19:03:02.300294+00:00 Aditi kernel: misc dxg: dxgk: dxgkio_query_adapter_info: Ioctl failed: -2
  • 2026-03-30T19:03:47.138755+00:00 Aditi kernel: WSL (242) ERROR: CheckConnection: getaddrinfo() failed: -5
  • 2026-03-30T19:13:27.918523+00:00 Aditi kernel: WSL (242) ERROR: CheckConnection: getaddrinfo() failed: -5
  • 2026-03-30T19:28:39.110215+00:00 Aditi python3[21936]: ["2026-03-30T19:28:39.109", "WARNING", "ubuntupro.system", "g
et_kernel_info", 166, "failed to process /proc/version_signature.", {}]
  • 2026-03-30T19:28:39.115190+00:00 Aditi python3[21936]: ["2026-03-30T19:28:39.114", "WARNING", "ubuntupro.system", "g
```

```
aditi@Aditi: ~

Keyword: failed
========================================

Analyzing log file...

========================================
ANALYSIS RESULTS
========================================
Keyword 'failed' found 78 times in /var/log/syslog

Total lines in file: 3565
Percentage of lines containing 'failed': 2.18%

========================================
LAST 5 MATCHING LINES
========================================
The last 5 lines containing 'failed':

  • 2026-03-30T19:03:02.300294+00:00 Aditi kernel: misc dxg: dxgk: dxgkio_query_adapter_in
  • 2026-03-30T19:03:47.138755+00:00 Aditi kernel: WSL (242) ERROR: CheckConnection: getad
  • 2026-03-30T19:13:27.918523+00:00 Aditi kernel: WSL (242) ERROR: CheckConnection: getad
  • 2026-03-30T19:28:39.110215+00:00 Aditi python3[21936]: ["2026-03-30T19:28:39.109", "WA
et_kernel_info", 166, "failed to process /proc/version_signature.", {}]
  • 2026-03-30T19:28:39.115190+00:00 Aditi python3[21936]: ["2026-03-30T19:28:39.114", "WA
et_kernel_info", 175, "Failed to parse kernel: 6.6.87.2-microsoft-standard-WSL2", {}]

========================================
Analysis Complete
========================================
aditi@Aditi:~$
```

Explanation :This script reads a log file line by line using the `while read` construct and counts the number of times a keyword such as "ERROR" appears using the `if-then` construct and the counter variable. It takes the filename as a command line argument using `$1`.

Script 5 — The Open Source Manifesto Generator

Code:

aditi@Aditi: ~

```
  GNU nano 7.2                                    manifesto_generator.sh *
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Description: Generates a personalized open source philosophy statement
# Alias concept: You can create an alias in ~/.bashrc: alias manifesto='./manifesto_generator.sh'

echo "========================================"
echo "Open Source Manifesto Generator"
echo "========================================"
echo "Answer three questions to generate your manifesto."
echo ""

# Question 1
echo "Question 1: Why do you love open source?"
read -p "Your answer: " Q1

# Question 2
echo ""
echo "Question 2: What open source project inspires you?"
read -p "Your answer: " Q2

# Question 3
echo ""
echo "Question 3: How will you contribute to open source?"
read -p "Your answer: " Q3

# Get name
```

aditi@Aditi: ~

```
  GNU nano 7.2                                    manifesto_generator.sh *

# Question 1
echo "Question 1: Why do you love open source?"
read -p "Your answer: " Q1

# Question 2
echo ""
echo "Question 2: What open source project inspires you?"
read -p "Your answer: " Q2

# Question 3
echo ""
echo "Question 3: How will you contribute to open source?"
read -p "Your answer: " Q3

# Get name
echo ""
echo "What's your name?"
read -p "Your name: " NAME

# Get current date
DATE=$(date "+%Y-%m-%d")

# Generate filename
FILENAME="manifesto_${DATE}.txt"
```

```
aditi@Aditi: ~
 GNU nano 7.2                                   manifesto_generator.sh *

 Get current date
ATE=$(date "+%Y-%m-%d")

 Generate filename
ILENAME="manifesto_${DATE}.txt"

 Create manifesto
at > "$FILENAME" << EOF
pen Source Manifesto - $NAME
ate: $DATE

 believe open source is about: $Q1

 am inspired by: $Q2

 will contribute by: $Q3

 commit to being part of the open source community and helping it grow.
OF

cho ""
cho "√ Manifesto created: $FILENAME"
cho ""
at "$FILENAME"
```

Output:

```
aditi@Aditi: ~
==========================================
Open Source Manifesto Generator
==========================================
Answer three questions to generate your manifesto.

Question 1: Why do you love open source?
Your answer: manifesto

Question 2: What open source project inspires you?
Your answer:  Python

Question 3: How will you contribute to open source?
Your answer: Help beginners contribute to their first project

What's your name?
Your name: adit

√ Manifesto created: manifesto_2026-03-30.txt

Open Source Manifesto - adit
Date: 2026-03-30

I believe open source is about: manifesto

I am inspired by: Python

I will contribute by: Help beginners contribute to their first project

I commit to being part of the open source community and helping it grow.
aditi@Aditi:~$
```

Explanation :This script uses the `read` command to ask the user three questions about their open source philosophy, then uses string concatenation and the `date`

command to create a personalized statement saved to a `.txt` file using the greater-than symbol (`>`). This is an example of user input, string concatenation, and writing to a file, and there is a comment about aliases.

CONCLUSION :

This capstone project is an all-encompassing audit of Python, an open-source software tool, in terms of its origin, licensing, ethical considerations, footprint in the Linux operating system, and more. Python was originally created by Guido van Rossum in 1989 as his "Christmas project" but is now arguably one of the most influential coding languages in the world, especially in AI and machine learning. Python is covered by the PSF License, which permits both individuals and corporations to utilize, modify, and disseminate Python freely. Using the five shell scripts created in this project, I have been able to apply my knowledge of Linux system administration. Moreover, I have been able to apply my knowledge of shell scripting with variables, loops, if statements, and command substitution. This project further reinforced my knowledge that open-source software tools like Python represent not only open-source tools but also an open-source philosophy—a philosophy that represents the foundation upon which modern technology is being developed.